Vulnerable Archive Security Analysis Report

This document provides a comprehensive security analysis of the “Vulnerable Archive” Django application. The application is intentionally designed with multiple security flaws to demonstrate both traditional web vulnerabilities and emerging risks introduced through Large Language Model (LLM) integrations. The goal of this report is to explain these vulnerabilities in a clear, human-centered way while highlighting their real-world impact and recommended mitigation strategies.

Overview

The application exposes several high-risk vulnerabilities across different layers, including backend logic, template rendering, external integrations, and deployment configurations. These issues collectively demonstrate how modern applications can become vulnerable not only through classic coding mistakes but also through unsafe AI integrations and poor operational practices.

Key Vulnerabilities and Analysis

1. Insecure Direct Object Reference (IDOR)

The application allows users to access, modify, or delete archives using only object IDs without verifying ownership. This means an attacker could manipulate IDs to access other users’ data. This type of vulnerability breaks access control and compromises data privacy.

2. SQL Injection

The search functionality builds SQL queries using raw user input. This allows attackers to inject malicious SQL code, potentially exposing or modifying sensitive data. It highlights the danger of bypassing ORM protections.

3. Stored Cross-Site Scripting (XSS)

User-generated content is rendered using unsafe template filters, allowing malicious scripts to be stored and executed in other users’ browsers. This can lead to session hijacking or data theft.

4. Unsafe LLM-to-SQL Execution

One of the most critical modern risks is allowing an LLM to generate and execute SQL queries directly. Without strict validation, the model can produce unsafe queries, leading to data leakage or destruction. This demonstrates how AI systems can amplify traditional risks if not properly constrained.

5. Arbitrary File Write / Path Traversal

The application allows file paths generated by the LLM to be used directly for writing files. This can enable attackers to overwrite sensitive files or write outside intended directories.

6. Server-Side Request Forgery (SSRF)

The system fetches URLs provided by users or generated by tools without validation. This can be exploited to access internal services or metadata endpoints, exposing sensitive infrastructure details.

7. Weak JWT Handling

The use of a hardcoded secret for token generation makes authentication predictable and insecure. Attackers could forge tokens and impersonate users.

8. Insecure Configuration

The application uses unsafe default settings such as DEBUG enabled and a fixed SECRET_KEY. These issues increase exposure in production environments.

	Vulnerability type	Where it showed up (original behavior)
1	IDOR (access others' archives by ID)	view_archive, edit_archive, delete_archive, enrich_archive
2	SQL injection	search_archives (string-built SQL with query)
3	Stored XSS	view_archive.html — notes / content with |safe
4	Unsafe LLM → SQL	ask_database — raw execution of model-generated SQL
5	Arbitrary file write / path issues	export_summary — LLM-chosen path, open(file_path, "w")
6	SSRF (server-side fetch)	add_archive — requests.get(url) to user URL
7	SSRF via tool / LLM	enrich_archive — fetch_url tool calling requests.get
8	Weak JWT handling	generate_token — hardcoded signing secret
9	Insecure deployment defaults	settings.py — fixed SECRET_KEY, DEBUG=True, tight ALLOWED_HOSTS

Fixes and Improvements

The vulnerabilities were addressed using standard security practices:

- Enforcing user-based access control to prevent IDOR.

- Replacing raw SQL queries with ORM-based filtering.

- Escaping all user-generated content to prevent XSS.

- Restricting LLM-generated queries to safe, read-only operations.

- Sanitizing file paths and restricting write locations.

- Blocking requests to private or internal network addresses.

- Moving secrets to environment variables.

- Hardening production settings and disabling debug mode.

Area	Problem	Fix
Archives	Users could open/edit/delete others' items by ID (IDOR)	Load archives with user=request.user so access is scoped to the owner.
Search	SQL injection from building raw SQL with the query string	Use the ORM (filter / icontains) instead of string SQL.
View page	Stored XSS from |safe on notes/HTML	Escape output: linebreaksbr for notes, plain <pre> for content; safer external links.
Ask database (LLM)	Running arbitrary SQL from the model	Allow only safe SELECTs on archiver_archive, block dangerous keywords, and force user_id so users only see their data.
Export summary	Path traversal / arbitrary file write from LLM-chosen paths	Write only under exported_summaries/ with a sanitized filename (no LLM path).
Add archive / enrich	SSRF via requests.get to internal/private URLs	Allow only public http(s) URLs and block localhost/private IPs (DNS resolved and checked).
JWT endpoint	Hardcoded JWT secret	Sign with settings.SECRET_KEY (from env in production).
Settings	Weak defaults (DEBUG, SECRET_KEY, ALLOWED_HOSTS)	Read from environment variables with safer defaults.

Testing and Validation

A suite of security-focused tests was executed to validate the fixes. These tests simulate real attack scenarios, ensuring that vulnerabilities such as IDOR, SQL injection, XSS, SSRF, and unsafe LLM behavior are properly mitigated. All tests passed successfully, confirming the effectiveness of the applied fixes.

Discussion: Limits of Exploit-First Testing

Exploit-first testing is highly effective for vulnerabilities that can be reproduced through predictable inputs and outputs. However, it falls short when dealing with secrets management and configuration issues. Problems such as leaked credentials or insecure environment variables are not always detectable through runtime tests.

A more robust approach includes:

- Secret scanning tools

- Secure CI/CD pipelines

- Environment-based configuration management

- Regular audits and monitoring

Conclusion

This application serves as a valuable case study in modern application security. It demonstrates how traditional vulnerabilities still persist while also introducing new risks through AI integration. The key takeaway is that security must be approached holistically, combining secure coding practices, proper configuration management, and careful control over AI-driven components.